

BladeView: Toward Automatic Wind Turbine Inspection With Unmanned Aerial Vehicle

Cong Yang¹, Hua Zhou, Xun Liu, Yan Ke, Bo Gao, Marcin Grzegorzec, Zeyd Boukhers², Tao Chen, and John See

Abstract—This paper presents a fully automatic method, BladeView, for drone-based wind turbine blade inspection using an Unmanned Aerial Vehicle (UAV). With the need for highly efficient blade inspection coupled with the rapid increase of wind turbines, existing methods provide limited automation on wind turbine parameter estimation, full blade coverage, and safety control. We introduce an Automatic Parameter Calculation (APC) algorithm and an Automatic Flight System (AFS) in BladeView to compute wind turbine parameters and inspection paths, respectively. Leveraging triangulation and linear fitting integration techniques, the APC automatically calculates the wind turbine parameters and estimates the relative angle and position between a drone and the turbine. Furthermore, with dynamic path finding and B-spline optimization, the AFS plans a path covering 3 blades within specified flight corridors, in compliance with the turbine parameters obtained from APC. Thus, the proposed BladeView can properly ensure an inspection's automation, coverage, safety, and smoothness. The efficiency and usability of BladeView are validated through 100,000 flight simulations in the Gazebo simulation environment and 9,239 field runs at various wind farms, including offshore, near-shore, deserts, mountainous areas, farmlands, and suburbs.

Note to Practitioners—The proposed BladeView is distinguished in three aspects: (1) It automatically adapts wind turbines with varying geometric properties and physical locations relative to the take-off point. (2) It dramatically improves the quality of collected data with optimal UAV speed and flight corridors. (3) It thoroughly covers all three blades of a Horizontal-Axis Wind Turbine (HAWT), including regions where defects frequently occur. Thus, BladeView is more efficient and robust than existing

UAV-based methods for blade inspection, with only around 25 minutes per HAWT. Moreover, it does not require experienced pilots to fly the UAV and manual interventions are rarely needed. Extensive simulation and real-world experiments demonstrate the efficiency and usability of BladeView in various on- and offshore wind farms.

Index Terms—Wind turbine, blade inspection, drone inspection, unmanned aerial vehicle, wind power.

I. INTRODUCTION

RECENTLY, camera and LiDAR-equipped Unmanned Aerial Vehicles (UAVs, hereafter referred to as “drones”) have been employed for wind turbine inspections (see Fig. 1 (c) and (d)) since they have obvious advantages in terms of efficiency, safety, and cost [1]. In particular, the main focus of interest lies on the Horizontal-Axis Wind Turbine (HAWT), as it produces the overwhelming majority of global wind power today [2]. For a better understanding, the main characteristics of drone-based wind turbine inspection are as follows:

- Each turbine has three blades, with each adjacent blade separated by 120 degrees.
- The blades stop spinning during the inspection process, and their final resting positions are usually close to ($\pm 30^\circ$) the shape of a vertically flipped “Y” (Fig. 1 (d)).
- Turbine geometric properties and specific physical location relative to the take-off point are most likely unknown (with minor information held from installation), though the drone is usually just in front of the wind turbine.
- Flying a drone equipped with a camera in close proximity to wind turbine blades is the core problem of the wind turbine inspection process.
- Weather conditions must allow for a safe flight while wind disturbances (at the height of >100 meters above the ground) could affect data quality.

In this paper, the scope of the problem focuses on flying a drone near wind turbine blades for a smooth, robust, and safe inspection, and ultimately collecting a complete set of high-quality blade images for defect analysis [3], [4]. To do so, the (multi-rotor) drone usually carries an on-board computer, camera, LiDAR, gimbal, and equipment for navigation (e.g., GPS, IMU, magnetic compass, barometer) [5]. A successful drone-based inspection is defined as one that is efficient, covers the full blade area, and safely collects the required images between take-off and landing [6], [7], [8], [9], [10]. During the inspection, manual operations should be avoided

Received 9 April 2024; revised 29 July 2024; accepted 16 September 2024. Date of publication 26 September 2024; date of current version 21 March 2025. This article was recommended for publication by Associate Editor C.-Y. Hsu and Editor B. Vogel-Heuser upon evaluation of the reviewers' comments. This work was supported in part by the Natural Science Foundation of Jiangsu Province under Grant BK20241918, in part by the National Natural Science Foundation of China under Grant 62473276, in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 22KJB520008, and in part by the Research Fund of Clobotics Research under Grant KB1801ZW201609-03. (Corresponding author: Tao Chen.)

Cong Yang and Tao Chen are with the School of Future Science and Engineering, Soochow University, Suzhou 215006, China (e-mail: chent@suda.edu.cn).

Hua Zhou, Xun Liu, Yan Ke, and Bo Gao are with Clobotics, Shanghai 200050, China.

Marcin Grzegorzec is with the Institute of Medical Informatics, University of Lübeck, 23562 Lübeck, Germany, and also with German Research Center for Artificial Intelligence, 23562 Lübeck, Germany.

Zeyd Boukhers is with the Fraunhofer Institute for Applied Information Technology, Sankt Augustin, 53757 Sankt Augustin, Germany, and also with the Institute of Biomedical Informatics, University Hospital of Cologne, 50937 Cologne, Germany.

John See is with the School of Mathematical and Computer Sciences, Heriot-Watt University, Malaysia Campus, Putrajaya 62200, Malaysia.

Digital Object Identifier 10.1109/TASE.2024.3464640

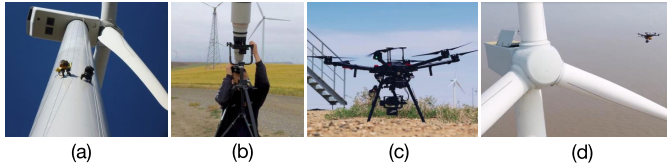


Fig. 1. Blade inspection: (a) rope technicians, (b) telescope, (c) drone. The scene of drone-based inspection is presented in (d).

due to the high cost of hiring experienced pilots and the low efficiency of manual control [11]. In practice, it is not only extremely difficult and unsafe but also time-consuming to manually stabilize the relative drone pose and its speed along the blades. As a result, pure manual operation on drone-based wind turbine inspection is challenging (based on current industrial practice from Clobotics Wind Service), mainly due to unstable images from inconsistent drone speed and distance. However, existing methods are limited in usability, primarily due to the lack of automatic *parameter estimation*, *full blade coverage*, and *safety control*.

A. Automated Parameter Estimation

Essential turbine parameters includes:

- **Turbine Height:** The height of the turbine hub center.
- **Turbine Position:** The position relative to the coordinate system of the drone take-off point.
- **Blade Stop Angle:** The angle between the turbine tower column and the first blade to its right (ideally 60°).
- **Turbine Heading:** The direction of the turbine rotor disk plane towards the nacelle (engine housing at the turbine hub rear).
- **Turbine Radius:** The radius at the turbine hub center.

Many existing methods cannot automatically estimate wind turbine parameters [12], [13], [14], [15], [16]. While wind turbines have varying geometric properties and physical locations relative to the take-off point, the basic parameters need to be estimated for automatic drone path planning. For instance, the height of the turbine hub, the orientation of the turbine blade plane, the blade stopping angle, and the local coordinates of the turbine tower relative to the drone take-off point (see definitions above). However, most existing methods have limited automation capabilities for these requirements. Though [13], [15] estimate the turbine pose using Hough transform [17] and neural networks [18], they are not robust and general enough for various on- and off-shore wind turbines [19].

B. Automated Full Blade Coverage

The three blades are not automatically and properly covered by existing methods [12], [20]. In practice, the drone path should cover all three blades, including the leading edge, pressure side, trailing edge, and suction side (the location of these parts are illustrated in Fig. 2). The inspection method in [11] only focuses on a single blade placed vertically, though it is generally extensible to full turbine inspection. The method in [15] does not have a requirement on the stopping angle, it only covers one blade per inspection (take-off, inspect, and return). The method in [14] covers three blades (mostly

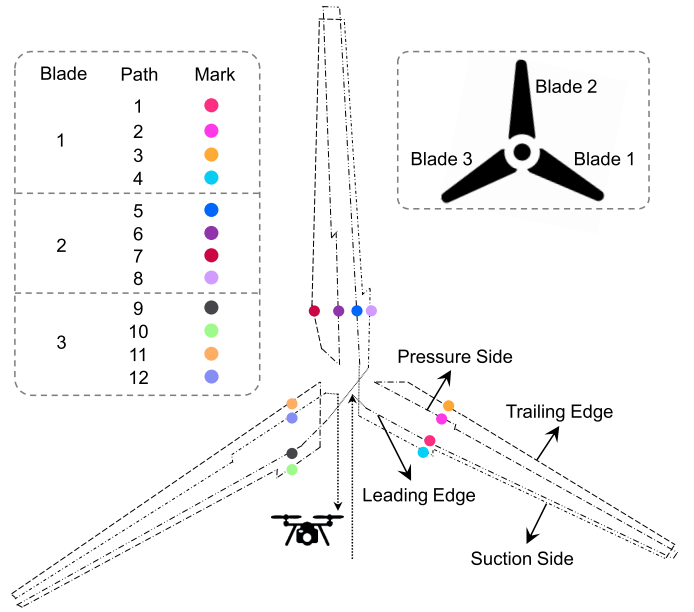


Fig. 2. The full inspection path (dot lines) of BladeView along the three blades. Each blade has four paths back and forth to cover the defect frequently occurred regions. Order: Blade 1, 2, and 3.

leading edges), but the inspection process is purely manual with arbitrary trajectories. It is difficult to ensure inspection efficiency and image quality.

C. Automated Safety Control

Existing methods have limited automation on safety control during inspection [1], [5], [16]. The impact of the wind forms a significant part of the disturbance, especially at wind farms where the annual average wind speed is 4.5 m/s and above (with gusts up to 10 m/s). The drone needs a certain level of wind resistance to avoid unnecessary overshoots and path oscillations. Moreover, the trajectory should be dynamically optimized to improve the smoothness and safety (e.g., maintaining a fixed drone-blade distance, hereafter referred to as “distance”) of inspection. Otherwise, as presented in Fig. 3 (top), the collected images are mostly unusable (blurred, uncovered, over/underexpose, transformed) due to unstable drone pose, speed, and distance.

Considering these challenges, we propose an Automatic Parameter Calculation (APC) method to automatically calculate the parameters of a wind turbine and to estimate the relative angle and position between a drone and the turbine. In addition, we propose an Automatic Flight System (AFS) for planning a path that covers all three blades and is optimized within dynamic flight corridors, in compliance with the turbine parameters calculated by the APC. In practice, both APC and AFS form a simple yet efficient method for automatic blade inspection, which we name BladeView. The rationale behind APC is that we integrate both triangulation and point cloud fitting algorithms to improve the accuracy and robustness of wind turbine parameter estimation. As such, the turbine parameters can be consistently measured under different environmental conditions, for example, when blades are stopped at any arbitrary (sub-optimal) angle.

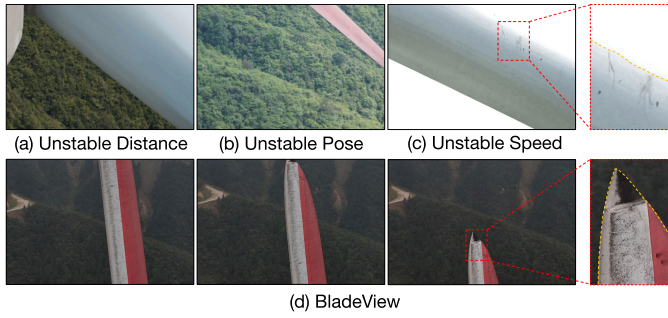


Fig. 3. Comparison of blade images collected by existing methods [16] (top) and BladeView (bottom). Due to unstable distance, pose, and speed measurements with existing methods, the collected images are mostly unusable. Images with BladeView have better quality and more easily distinguished features: centered in the image, full blade coverage, clear blade edge, easy to segment and stitch.

Regarding AFS, it automatically plans a three-blade-covered path based on the turbine parameters from APC. Notably, all parts of each blade are adequately covered (see Fig. 2). We also propose generation of flight corridors along the determined path to ensure flight safety. Moreover, an optimal path finding algorithm is designed to stabilize the distance and speed of inspection for wind disturbances. As presented in Fig. 3 (d), the quality of collected images can be dramatically improved if the smoothness of an inspection operation can be ensured.

Succinctly, the main contributions are: (1) we introduce a drone-based inspection method, BladeView, for automatically inspecting wind turbine blades with varying geometric properties and physical locations relative to the take-off point. To accomplish this, we propose APC and AFS to calculate wind turbine parameters and inspection paths, respectively. Compared with existing methods, the APC is more robust, and flexible for both onshore and offshore wind turbines. Meanwhile, the AFS ensures inspection safety, efficiency, and full blade coverage with optimized drone speed, distance, and path within flight corridors. (2) BladeView has been validated through over 100,000 flight simulations in the Gazebo simulation environment [21] and over 9,000 real-world flight tests (field runs) at various wind farms and environments, including deserts, mountainous areas, farmlands, suburbs, near-shore, and offshore. It should be emphasized that BladeView can not completely replace pilots, especially BladeView remains subject to unanticipated errors that can result in damage to the drone or wind turbine necessitating manual intervention (see Section IV-E). Therefore, in this paper, we underscore the suitability of BladeView for unmanned inspection. In particular, a safety pilot is still required for equipment transport, inspection activation, and emergency control. We hope BladeView will benefit future research in the area of inspection, particularly in addressing practical and real-world concerns.

As shown in Fig. 4, the rest of the paper is structured as follows. Section II discusses related works on wind turbine inspection and correlated autonomous navigation and control algorithms. Section III details our proposed APC and AFS in BladeView. The efficiency of APC, AFS, and BladeView are quantitatively validated in Section IV in both simulated and real-world environments. Section V closes the paper with conclusions and insights into our future research plans.

II. RELATED WORKS

In this section, we briefly review several existing drone-based wind turbine inspection methods, followed by a review of related autonomous navigation and control methods. For a more detailed treatment of these topics in general, the recent compilations by Kulsinskas et al. [20] and Nooralishahi et al. [5] also offer good insights. For better understanding, we first clarify the standards and requirements of wind turbine blade visual inspection. Precisely, drone-based blade inspection captures high-resolution blade data (mostly image sequences) with fixed trajectories along the blades. Defect detection, deduplication, and full-blade visualization are essential in blade maintenance systems, contributing significantly to wind turbine status monitoring, inspection scheduling, and repair guidance. To ensure the full coverage of blades (ranging from 20 to 80 m) and high resolution of defects (≤ 0.4 mm/pixel), blade image sequences are usually captured along the flight trajectory. To aid defect analysis and repair preparation, displaying an entire blade in multimedia applications is proper, along with clear markings of defect types and regions. In blade maintenance systems, the original image size is around 5400×3600 pixels, reflecting a gigapixel-size full visualization after stitching. A surface defect is a reliable indication of structurally (or functionally) significant damage accrued over the operational lifespan of the wind turbine blade. Mainly, defect types include LPS (Lighting Protection System) missing, LPS burnt, VG (Vortex Generator) missing, VG broken, drain hole blocked, serration missing, serration damage, tip damaged, LEP (Leading Edge Protection) erosion, leading edge erosion deep, leading edge erosion hole, tip erosion, surface burn damage, surface crack, and trailing edge crack, etc. Contaminations include birds, aviation fuel (oil stains), smudges, attached sand and insects, etc.

A. Wind Turbine Inspection

While existing methods mainly concentrate on off-line blade defect detection using deep learning approaches [22], [23], [24], few studies have been devoted to efficient wind turbine inspection using drones. As pioneers, Stokkeland et al. [13] and Wang and Zhang [14] presented several vision-based methods for measuring turbine parameters. However, most of them are not as robust in practice since ambient light variations badly affect measurement accuracy. Recently, Car et al. [1] introduced an inspection method using LiDAR-equipped drones to collect high-resolution blade images and 3D models of wind turbines. This method is more robust than purely vision-based since LiDAR is less affected by light than cameras. However, the usability of this method still needs to be improved. Firstly, it only covers one single blade rather than the entire turbine (though it has the capability of extending to three blades). Moreover, the pressure and suction sides of the blade (see Fig. 2) are not covered. Thirdly, the employed Proportional-Integral-Derivative (PID) algorithm [25] is sensitive (e.g., overshoots and oscillates) to gust wind disturbances. Our experiments in Section IV-D show that the PID algorithm has only around 90% success rate in the drone-based wind turbine inspection scenario. Here, success is defined as a

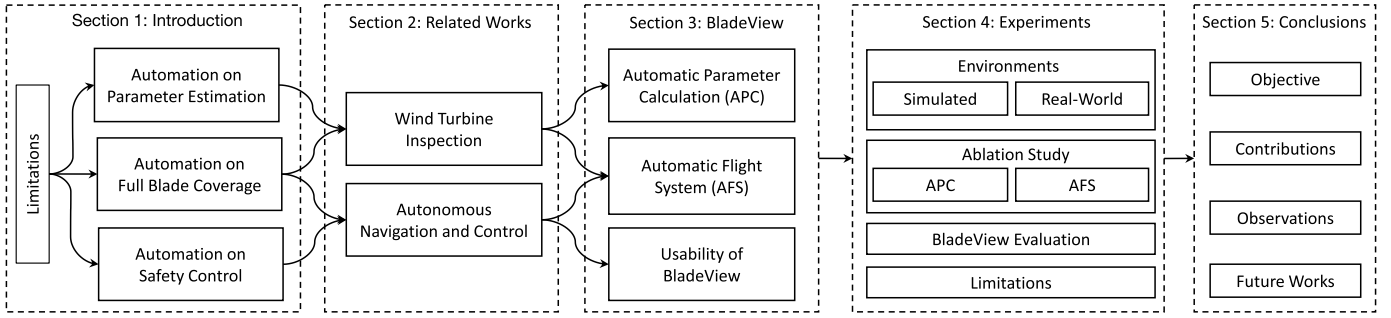


Fig. 4. Paper structure and the main contents in each section.

successful end-to-end wind turbine inspection, respecting both path integrity and image quality. It should be noted that wind gust rejection depends on multiple factors other than flight control algorithms, such as rotor design, gravity, tilt outwards, onboard flow sensing devices, etc. For a more detailed treatment of this topic in general, the recent compilations by Simon et al. [26] offer sufficiently comprehensive insights. Unlike existing methods, our proposed BladeView is an end-to-end and automatic method for full turbine blade inspection. Notably, we introduce the APC method to measure the turbine parameters automatically. To ensure a constant drone-blade distance for safety, we also present the AFS method to generate an optimized inspection flight path within dynamic corridors. Its usability and high success rate are validated (see Section IV) in various simulated and real-world environments.

To improve the blade coverage and inspection efficiency, Mansouri et al. [27] proposed to inspect a wind turbine using a slicing-intersection-clustering-offset adding strategy. The inspection trajectory is a set of cylinders following the waypoints around tubular steel and blades. The collected images are post-processed for 3D reconstruction using SfM. Though this approach can guarantee the full coverage of three blades, its usability is still limited: (1) The trajectory is well suited to reconstruction. However, it has potentially high collision risks, especially at the turbine hub. Maintaining stable inspection distance and speed along the cylinder trajectories is highly challenging in practice. (2) The turbine hub and its connection regions to the three blades are not fully covered. (3) As required by wind farm management systems, the collected images from cylinders cannot be used for stitching and high accurate defect deduplication in pixel level. Kanellakis et al. [28] introduced an algorithm for full coverage inspection path planning of wind turbines and other branching structures. The inspection path is formed of cylinders located around the turbine. Differently, inspection trajectories generated using BladeView are simpler (being close to a straight line along the leading edge), cover regions prone to significant defects, are shorter, and easier to navigate via a smooth inspection path for high-quality defect screening.

Beyond wind turbine blades, Bircher et al. [29] proposed to inspect a given surface (manifold in known and unknown volume) in a receding horizon fashion by sampling possible future configurations in a geometric random tree. Though the entire surface can be covered in both indoor and outdoor environments, the inspection (or exploration) path is unstable and

randomly distributed. Similarly, Jing et al. [30] introduced a coverage path planning method using path primitive sampling and primitive coverage graph. Guerrero and Bestaoui [31] proposed a UAV path planning method for structure inspection by considering wind influence and onboard energy limitation. They derived three methods to inspect a structure, ranging from route computation to time-optimal route minimization based on detected interest points. The method was validated in a simulated environment on bridge column inspection. Recently, Jing et al. [32] introduced a multi-UAV coverage path planning framework for inspection, a typical multi-agent path planning problem. They converted this problem into two NP-hard sub-problems (the Set Covering Problem and the Vehicle Routing Problem), solved by a modified Biased Random Key Genetic Algorithm. The method was validated via six building models in a simulated environment.

However, since the inspection paths are varied in field runs using these methods, their usability is limited in blade inspection scenarios: (1) it is unnecessarily complex on wind turbines with standard structures and rigid blades. Notably, it is challenging for pilots to predict risks (for taking over) with a random inspection path. (2) Image quality and overlap are not ensured. Particularly, the yield of images of suitable quality for stitching and deduplication is very low, leading to an inefficient inspection process. (3) The inspection time cannot be precisely estimated, which is unacceptable in commercial inspection. Differently, the inspection path is simple and predefined in BladeView, which is friendly for pilots to assess the inspection status. The inspection time is guaranteed with BladeView: around 25 minutes per wind turbine, 31 onshore and 18 offshore wind turbines per day (see Section IV).

In addition to academic research, we report numerous commercial inspection systems with autonomous capability. Drones from Skydio [7] are used for bridge inspection, telecom asset inspection, and construction site monitoring. It can also be extended for blade inspection with BladeView. DJI 3DX Blade Platform from Sulzer Schmid [8] provides an end-to-end drone-based inspection solution for blade asset management, including planning, capture, processing, annotation, analytics, and action. It ensures 100% blade coverage with high-resolution image capture. Perceptual Robotics [9] also provides customised automation for intelligent onshore and offshore wind turbine inspection solutions, ranging from drone-based high-resolution blade data acquisition and processing to a web portal for inspection progress tracking.

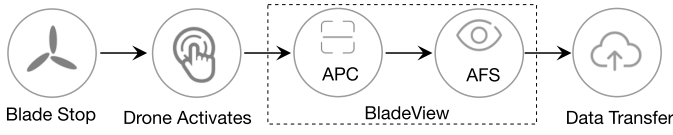


Fig. 5. Full pipeline of drone-based blade inspection. Our proposed BladeView is composed of APC and AFS.

Similarly, SkySpecs [10] autonomous drone inspections capture blade data and recognize blade defects, thereby identifying blade trends to mitigate risks. At the time of paper writing, the detailed algorithms, parameters, and performances of field run tests of the above products are unknown.

B. Autonomous Navigation and Control

To yield a smooth and safe trajectory for drone-based inspection, Vanneste et al. [33] introduced an obstacle avoidance method that both global and local planners are bounded. For navigation, it uses the current position and the obstacle information from LiDAR for map updating. While using the A* algorithm [34] to plan an optimal path, the dynamic and kinematic constraints of the drone are not considered. To solve this problem, Lopez and How [35] proposed a fast local planner method based on the kinematic behaviour of a drone and the initial movement of A*. Depending on the relative positions of obstacles and the drone, the space is divided into danger and safety zones (aka. flight corridors), according to their distances. However, the planned path must be improved to keep the drone safe and efficient due to possible control errors from environmental disturbances and actuator saturation. For this, Zhou et al. [36] proposed a method that combines kinodynamic path searching, B-spline optimization, and time adjustment. As a result, the trajectory is smooth, safe, and dynamically feasible, proven by simulation and real-world tests. However, this method is inefficient for the onboard computer due to the high memory cost of the ESDF (Euclidean Signed Distance Field) map [37]. Gooijaers [38] also proposed a path-planning method for wind turbine inspection which is optimized based on Dijkstra's algorithm. However, it does not consider the restrictions from kinematics and Newton's laws of motion. As a result, the planned path requires additional smoothing for practical usage. Besides, the robustness of this method is not guaranteed, as it was not thoroughly evaluated in various real-world environments.

Compared with [33], [35], and [36], our method is more robust and suitable for wind turbine blade inspection. In addition to full-blade coverage and flight corridors for safety, BladeView has a certain level of wind resistance for stable speed and distance based on B-spline fitting and OctoMaps [39], which has lower memory usage than the ESDF map.

III. BLADEVIEW

Fig. 5 presents the drone-based inspection pipeline, from blade stop to the final transfer of inspection data (usually to a cloud-based system for further blade stitching and defect analysis [22], [40]). Our proposed BladeView mainly focuses on the core inspection part after drone activation. For this, it is

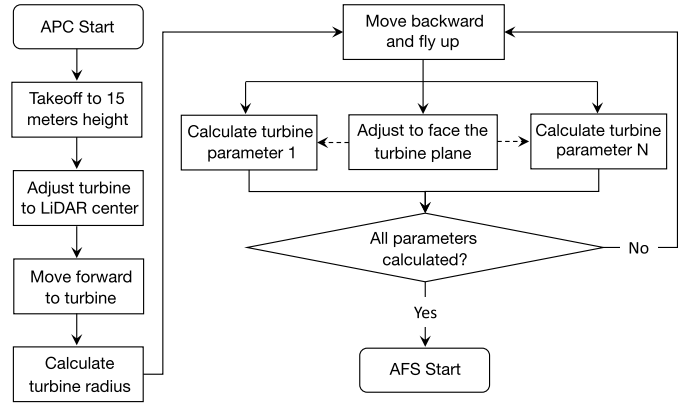


Fig. 6. Flow chart of APC for the measurement of turbine parameters.

TABLE I
CRUCIAL SYMBOLS AND THEIR DESCRIPTIONS INVOLVED IN SECTION III

Symbol	Description	Subsection
Δ_{result}	Output from APC	A
$\Delta_{triangle}$	Measured parameters from triangulation	A
Δ_{linear}	Measured parameters from linear fitting	A
ω_0, ω_1	Weight of two APC terms	A
h_{cor}	allowable error of the drone distance	A
$p(t)$	Drone position (in 3D space) at time t	B
$v(t)$	Drone speed at time t	B
v'	Pre-defined target speed	B
$a(t)$	Drone acceleration at time t	B
$\mathbf{x}$	All three drone state quantities	B
$\dot{\mathbf{x}}$	Differential of $\mathbf{x}$	B
$\mathbf{u}$	Control input in path finding	B
$-U_{max}$	Lower extreme value of control variables	B
U_{max}	Upper extreme value of control variables	B
f_{cost}, g_{cost}	Path and control costs	B
h_{cost}	Heuristic value	B
d_{cost}, v_{cost}	Distance and speed costs	B
$\lambda_1 - \lambda_4$	Weights to balance the four costs	B
$\mathbf{p}$	Control point of B-spline	B
Δt	Knot span of B-spline	B
pb	Degree of piecewise polynomial	B
f_{total}	Overall cost of B-spline	B
f_s and f_d	Smoothness and distance to collision	B
f_v and f_a	Soft limits on velocity and acceleration	B
d_{min}	The smallest distance for a safe inspection	B

necessary that we measure the parameters of a wind turbine and estimate the drone-turbine relative position using the APC module. Following that, the AFS module is used to plan the three-blade-covered path based on the empirical definition (see Fig. 2) and parameters from APC. For safety, AFS generates flight corridors along the path to avoid potential collisions. The inspection trajectory is also dynamically optimized to ensure stable and smooth inspection (free from wind disturbances) based on the cost functions in AFS. For better understanding, some essential symbols are summarized in Table I.

A. Automatic Parameter Calculation (APC)

Fig. 6 presents the general idea of APC. It automatically measures the turbine radius and other parameters until AFS is ready to start. APC is motivated by the fact that the state of the turbine shutdown has certain randomness, which makes

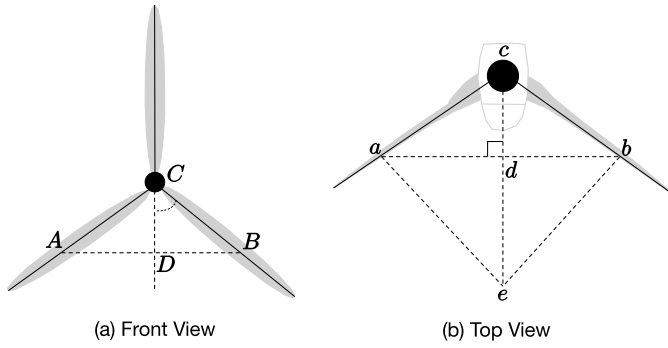


Fig. 7. General idea of the triangulation algorithm. Top view in (b) means vertically downward, thereby one blade is invisible.

it difficult for the AFS to perform the automatic inspection. Moreover, pursuing an ideal blade stop position is inefficient and almost impractical. Thus, BladeView's APC acts to provide tolerance for a wind turbine with a sub-optimal angle, various geometric properties, and physical locations relative to the take-off point. Formally, let Δ_{result} denote the final output from APC. It contains five parameters (Turbine Height, Turbine Position, Blade Stop Angle, Turbine Heading, and Turbine Radius) defined in Section I.

The Turbine Radius is calculated based on the estimated circles from LiDAR point clouds with the random sample consensus (RANSAC) algorithm [41] (see Fig. 6). As a by-product, the Turbine Position is collected from the center of the circles. To evaluate the remaining three parameters in Δ_{result} , triangulation and linear fitting algorithms are adopted by the APC. Specifically, let $\Delta_{triangle}$ and Δ_{linear} denote the measured parameters from the triangulation and linear fitting algorithms, respectively. Δ_{result} is calculated by:

$$\Delta_{result} = \omega_0 \Delta_{triangle} + \omega_1 \Delta_{linear}. \quad (1)$$

where ω_0 and ω_1 are the weights to balance the contribution of each term. In practice, we empirically set $\omega_0 = 0.2$ and $\omega_1 = 0.8$ based on our preliminary experiments in the simulator, followed by verification in our field runs of seven environments (see Table II). The rationale behind Eq. 1 is that both methods are employed to improve the robustness of APC. It should be noted that a more optimised fusion of the relevant performance aspects from the linear and triangular parameter estimation methods (as an avenue for future research) may benefit the outcome, as the individual methods errors are in some cases superior to the combined result (see Table III).

1) *Triangulation Algorithm*: Once the tower column, left and right blades are detected, their triangle relationship is formulated. Fig. 7 presents such a relationship in front and top views. In Fig. 7 (a), C is the hub center, A and B are the LiDAR point clouds of the left and right blades, and D is the projection of the drone's location. CA and CB are the left and right diagonal blades, and CD is the projection of the vertical tower on the plane ABC . Fig. 7 (b) presents the top view of (a). Particularly, a , b and c are corresponding to A , B and C in Fig. 7 (a), e is the real position of the drone, while d is the projection of the actual position (like D in (a)) where a , b and d are collinear. Thus, $ad = AD$, $db = DB$. ACB is a

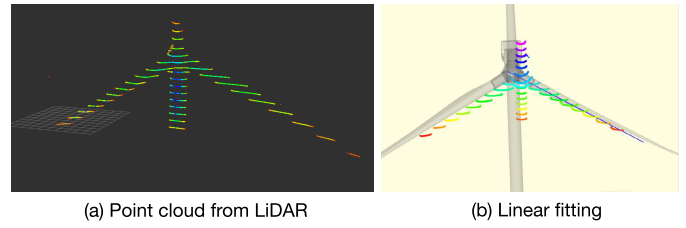


Fig. 8. Linear fitting result (the blue straight line in (b)) with the point cloud (consecutive frames) from LiDAR.

standard isosceles triangle, and $\angle ACB = 120^\circ$ based on the inherent HAWT characteristics. Suppose the drone faces the rotor disk's plane, $ab \perp cd$. Otherwise, the drone needs to be adjusted (see Fig. 6). To calculate the parameters in $\Delta_{triangle}$, $\angle BCD$ is defined as the Blade Stop Angle when the turbine shuts down and its blades cease to move:

$$\angle BCD = \frac{1}{2} \left[\sqrt{\frac{(n+1)^2}{3n^2} + \frac{4}{n} + (n+1)\sqrt{\frac{n}{3}}} \right]. \quad (2)$$

where $n = ad/bd$. This is because $ad = AD$, $db = DB$, and based on their triangle relationships:

$$\frac{\tan \angle BCD}{\tan \angle DCA} = \frac{BD}{AD}. \quad (3)$$

where $\angle DCA + \angle BCD = 120^\circ$ according the characteristics of HAWT. By Eq. 2, CD is calculated as:

$$CD = \frac{BD}{\tan \angle BCD}. \quad (4)$$

based on the trigonometric principle and the front view relationship of $\tan \angle BCD = BD/CD$ and $\tan \angle ACD = AD/CD$. Let H_D denote the height of D (same to drone's height), therefore the Turbine Height is $H_D + CD$. The Turbine Heading is determined by the yaw of the plane.

The triangulation algorithm is simple and efficient, it only needs LiDAR to detect the point clouds of the tower, the left and right blades. Then, the center point coordinates are analyzed through point cloud clustering to obtain a more accurate $\Delta_{triangle}$. However, the accuracy of $\Delta_{triangle}$ highly depends on the stop angle $\angle BCD$. Notably, while the deviation increases from 60° , points A and B in Fig. 7 (a) could be slightly shifted. This is because the point cloud area of the diagonal blade will increase, further affecting the point cloud clustering results. To solve this problem, a linear fitting algorithm is introduced.

2) *Linear Fitting Algorithm*: As presented in Fig. 8 (a), we first use the clustering algorithm [42] to separate point clouds (several consecutive frames) from LiDAR into three blades. Then, we fit a straight line to the point cloud representation of each blade (see Fig. 8 (b)). This is motivated by the fact that point clouds of blades represent line features. In BladeView, we use the RANSAC algorithm [41] for line fitting. Precisely, within a fixed number of times, we iteratively and randomly choose hypothetical inliers from the point clouds to estimate lines. The line that fits most inliers is selected as the blade fitting line. To improve the robustness of this process, we filter the point clouds by the Extended Kalman

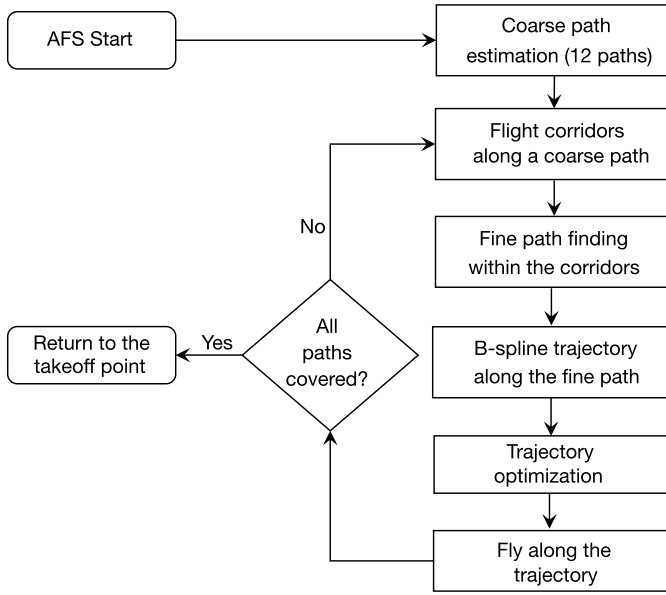


Fig. 9. Flow chart of automatic flight system (AFS).

Filter (EKF) [43] to eliminate interference points. Finally, the parameters in Δ_{line} are calculated based on the slope of the lines.

B. Automatic Flight System (AFS)

As presented in Fig. 9, the turbine parameters Δ_{result} are first used for coarse path estimation. Specifically, the turbine radius, turbine position, and drone position are used to estimate the plane coordinate of the wind turbine. Together with the turbine height, the coordinates of the rotor disk (the black point in Fig. 7(a)) are first localized. Then, using the rotor disk as an anchor, the coarse paths (see dot lines in Fig. 2) are estimated according to the turbine heading and blade stop angle in 3D space. In particular, the coarse paths (12 paths) are empirically defined by blade maintenance experts to ensure the full coverage of defect prone regions (Pressure Side, Trailing Edge, Leading Edge and Suction Side). During the inspection process, the pre-defined coarse path is used to extract the medial axis of flight corridors (aka. Flight corridors along a coarse path in Fig. 9). To do so, the coarse paths are generated based on the rotor disk coordinates, estimated blade length, prior knowledge of HAWT, and pre-defined path order. AFS iteratively estimates a straight line along the blade during the inspection based on our solid-state LiDAR (FOV: 120°). The straight line is then translated to the propeller within the corridor and treated as a trajectory refinement. The refinement is around 10 Hz (0.1 seconds per frame), and the overall trajectory is stable and smooth in practice. With the estimated rotor disk coordinate (relative to the drone) from APC, as presented in Fig. 2 (top-right), AFS starts the blade inspection with the order of 1, 2, and 3. In practice, the blade inspection order is adjustable. Once the first blade inspection is finished, the rest of the paths are encoded based on the estimated first blade length (same as Blade 2 and 3), the prior knowledge of HAWT (120° between adjusted blades), and the pre-defined coarse path order (see Fig. 2 (top-left)). Consequently, the 12 paths

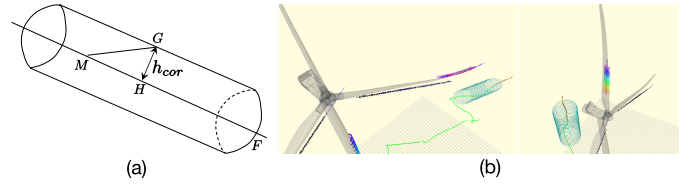


Fig. 10. (a) Flight corridor primitive. (b) The center line of cylinder is dynamically updated based on the fitted B-spline.

adequately cover all three blades, including both leading and trailing edges, as well as pressure and suction sides of each blade. In other words, the regions where defects frequently occur are fully covered by the inspection trajectory.

The coarse paths serve as the basis for AFS trajectory refinement. During the inspection process, the pre-defined coarse path is used to extract the medial axis of flight corridors (aka. Flight corridors along a coarse path in Fig. 9). To do so, AFS iteratively estimates a straight line along the blade during the inspection based on the solid-state LiDAR. The straight line is then translated by a set distance to define the centre of a flight corridor and treated as a trajectory refinement. We further introduce the Flight Corridor and B-spline methods along the coarse path to find a finer trajectory for a safe and smooth inspection. Note that it is also possible to control the drone without trajectory planning. In such a case, the inspection path may not meet the aircraft's dynamic constraints and hence cannot be restored to the target position immediately under the influence of wind. As a result, it could affect the inspection safety due to the missed target. Further details of the AFS are presented as follows:

1) *Flight Corridor*: Fig. 10 presents the general idea of the flight corridor. M , F , and G are the start, target, and path control points, respectively. H is the projection of G on the center line. The flight corridor h_{cor} denotes the tolerance (aka. allowable error) of the drone distance calculated by:

$$h_{cor} = |\vec{MG}| \sin(\cos(\frac{\vec{MG} \cdot \vec{MF}}{|\vec{MG}| |\vec{MF}|})). \quad (5)$$

where $|\vec{MG}|$ indicates the norm of the vector $\vec{MG}$. $\vec{MG}$ and $\vec{MF}$ are calculated from LiDAR point clouds. It should be mentioned that in practice, the blade tip part is normally curved (see the rightmost blade in Fig. 10 (b)) due to its inherent design. In such a case, rather than a straight line, the slope of the blade changes gradually. Thus, the target point F should be dynamically updated to ensure a stable drone-blade distance. Our proposed APC is inherently robust to the blade curvature since the linear fitting algorithm in AFS can dynamically estimate the blade slope.

To generate flight corridors, the ESDF (Euclidean Signed Distance Field) [36], [37], [44] maps have been actively used for motion planning. However, they have inherent shortcomings: (1) The map and grid sizes must be set in advance. (2) Large maps consume memory space and are unsuitable for onboard computing. For this, we build a localized OctoMap [39] which follows the size of the flight corridor (with the same size). This strategy can effectively reduce memory usage since the corridor's flight trajectory is relatively

fixed: the points on the cylindrical surface are marked as obstacles while the interior is accessible. According to the drone's current state, the local OctoMap is established with the drone as the center, including the distance and gradient information of the drone to the obstacles.

2) *Path Finding*: We employ the hybrid A* method [34] to find a safe and optimal path within the grid map of corridors. Particularly, the kinematics and dynamics constraints of the drone are considered in the AFS. Let $p(t)$, $v(t)$, and $a(t)$ denote the drone position (in three-dimensional space), speed, and acceleration at time t , respectively. Let $\mathbf{x}$ succinctly denote all three state quantities:

$$\mathbf{x}(t) = [p(t), v(t), a(t)]^T. \quad (6)$$

According to the kinematics relationship, velocity $v(t)$ and acceleration $a(t)$ satisfy the dynamic constraints: $\|v(t)\|_\infty \leq V_{max}$, $\|a(t)\|_\infty \leq A_{max}$, where V_{max} and A_{max} are the maximum value of speed and acceleration, respectively. With the initial state value $\mathbf{x}(0)$, and the control input $u(t)$, the system state $\mathbf{x}$ at any moment t can be calculated via the dynamic model:

$$\dot{\mathbf{x}} = K_1 \mathbf{x} + K_2 \mathbf{u} \quad (7)$$

$$K_1 = \begin{bmatrix} 0 & \mathbf{I}_3 & 0 & \cdots & 0 \\ 0 & 0 & \mathbf{I}_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \cdots & 0 & \mathbf{I}_3 \\ 0 & \cdots & \cdots & 0 & 0 \end{bmatrix}, \quad K_2 = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \mathbf{I}_3 \end{bmatrix}.$$

where $\dot{\mathbf{x}}$ is the differential of $\mathbf{x}$, K_1 and K_2 are the coefficient matrices describe the dynamic behaviour of the drone and its response to control input. $\mathbf{I}_3$ is a 3×3 matrix empirically defined based on our preliminary experiments. For instance, we set $\mathbf{I}_3 = [1, 0, 0; 0, 1, 0; 0, 0, 1]$ for the experiments in Section IV. $\mathbf{u}$ is the control input, including locations in 3D space and angular accelerations in 3 directions. These parameters can be converted into the vector thrust of each thrower steering gear with, e.g., quadcopter dynamics equation [45]. In practice, $\mathbf{u}$ is divided into equal distances as:

$$\mathbf{u} = \left[-U_{max}, -\frac{c-1}{c}U_{max}, \dots, \frac{c-1}{c}U_{max}, U_{max} \right]. \quad (8)$$

where $-U_{max}$ and U_{max} are the lower and upper extreme values of the control variables and c denotes the number of distance intervals divided from $\mathbf{u}$. We empirically set $c = 5$ in our experiments. Here, we can also fix the time expansion change as Δt to get the whole possible status. In our case, $\Delta t = 0.1$.

From $\mathbf{x}(t)$, our target is to find the best path to $\mathbf{x}(t+1)$ that is optimal in time and control, as defined by the path cost:

$$f_{cost} = \lambda_1 * g_{cost} + \lambda_2 * h_{cost} + \lambda_3 * d_{cost} + \lambda_4 * v_{cost}. \quad (9)$$

where g_{cost} , h_{cost} , d_{cost} , and v_{cost} are the control cost, heuristic value, distance cost, and speed cost, respectively. To avoid dominant values and to ensure relatively equal contributions, λ_1 , λ_2 , λ_3 , and λ_4 are weights to balance these four costs. In our simulated environment, the weights are initially optimized at a coarse-grained level, followed by fine-tuning in

real-world environments (see Section IV-A1 and IV-A2 for more details). We select $\lambda_1 = 1$, $\lambda_2 = 5$, $\lambda_3 = 70$ and $\lambda_4 = 10$ in our experiments. Following the terminology of A*, g_{cost} represents the actual cost of an optimal path from the start state to the current state. An admissible and informative heuristic is essential to speed up the search in A*. As described in [36], the control cost g_{cost} is computed by the cost of motion primitives (from start to current) generated with the discretized input and duration. The heuristic cost h_{cost} is calculated by applying the Pontryagins minimum principle [46] among the current and goal position and velocity. d_{cost} is calculated by $d_{cost} = h_{cor}^2$ (cf. Eq. 5 and Fig. 10 (a)), v_{cost} represents the gap between current $v(t)$ and target speed v' , i.e. $v_{cost} = \|v(t) - v'\|$. In practice, v' is pre-defined before take-off. For instance, we normally set $v' \in [1, 3]$ (m/s) in our flight tests (cf. Section IV).

3) *B-Spline Trajectory*: Based on the path finding in Section III-B2, the initial path meets the basic flight requirements of the drone: move to the target point and avoid obstacles. However, the drone is still likely to collide with obstacles if there are environmental interference and control errors. To optimize the path and to obtain a feasible and safe trajectory, we use B-spline curve [36] to fit the path. The B-spline curve is a piecewise polynomial, uniquely defined by: (1) degree pb ($pb = 3$ in our case), (2) a set of control points $\{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_N\}$, and (3) a knot vector $[t_0, t_1, \dots, t_M]$. Theoretically, a B-spline trajectory is parameterized by time t , where $t \in [t_{pb}, t_{M-pb}]$, $M = N + pb + 1$. For a uniform B-spline, each knot span $\Delta t_m = t_{m+1} - t_m$ has the same value Δt ($\Delta t = 0.1$ in our case).

Inspired by [36], to evaluate the position at time $t \in [t_m, t_{m+1}] \subset [t_{pb}, t_{M-pb}]$, t is firstly normalized as $s(t) = (t - t_m) / \Delta t$. Based on that, the matrix representation method in [47] is employed to evaluate the drone position $p(s(t))$:

$$p(s(t)) = \mathbf{s}(t)^T M_{pb+1} \mathbf{q}_m$$

$$\mathbf{s}(t) = [1, s(t), s^2(t), \dots, s^{pb}(t)]^T$$

$$\mathbf{q}_m = [\mathbf{p}_{m-pb}, \mathbf{p}_{m-pb+1}, \mathbf{p}_{m-pb+2}, \dots, \mathbf{p}_m]^T. \quad (10)$$

where M_{pb+1} is a constant matrix determined by pb . While the derivative of a B-spline is also a B-spline, the evaluation of the derivatives are the same. Here, the convex hull property of B-splines is essential for designing our optimization method described in the next section.

4) *B-Spline Optimization*: To optimize the B-spline from Section III-B3, the overall cost f_{total} is defined as:

$$f_{total} = \sigma_1 f_s + \sigma_2 f_d + \sigma_3 f_v + \sigma_4 f_a. \quad (11)$$

where f_s and f_d are the smoothness and distance to collision. f_v and f_a are soft limits on velocity and acceleration. σ_1 , σ_2 , σ_3 and σ_4 are weights to provide a trade-off between the smoothness, safety (distance) and dynamic feasibility (speed and acceleration). Similar to Eq. 9, the weights are empirically optimized from coarse to fine in our simulated and real-world environments, respectively. We finally select $\sigma_1 = 1000$, $\sigma_2 = 10$, $\sigma_3 = 100$ and $\sigma_4 = 100$ in our experiments. Note that most notations in this part are predefined in Section III-B2 and III-B3.

Smoothness cost f_s captures the geometric property of the trajectory. Ideally, if f_s equals zero, all the control points are uniformly distributed in a straight line [48], [49]. In this paper, f_s is calculated by:

$$f_s = \sum_{i=(pb-1)}^{(N-pb+1)} \|(\mathbf{p}_{i+1} - \mathbf{p}_i) + (\mathbf{p}_{i-1} - \mathbf{p}_i)\|^2. \quad (12)$$

where the variable i traverses from $(pb-1)$ to $(N-pb+1)$ to avoid line start and end points since these two points are already predefined. The collision cost f_d is formulated as the repulsive force of the obstacles on each control point:

$$f_d = \sum_{i=pb}^{(N-pb)} F_c(dist)$$

$$F_c(dist) = \begin{cases} (d(\mathbf{p}_i) - d_{min})^2 & \text{if } d(\mathbf{p}_i) > d_{min} \\ 0 & \text{otherwise.} \end{cases} \quad (13)$$

where d_{min} denotes the smallest distance for a safe inspection. Our experiments set d_{min} to 1 meter. $d(\mathbf{p}_i)$ is the distance between $\mathbf{p}_i$ and the center line of flight cylinder corridor, calculated similarly in Eq. 5. The rationale behind f_d is that the drone flight corridor must be limited within a certain range; thereby, the drone and turbine blade are kept within a predetermined distance. In terms of the drone velocity f_v , it is also limited to a certain range to ensure the image quality remains acceptable (blurred otherwise):

$$f_v = \sum_{\xi} \sum_{i=(pb-1)}^{(N-pb)} F_v(V_i)$$

$$F_v(V_i) = \begin{cases} (|v(i)| - V_{max})^2 & \text{if } |v(i)| \geq V_{max} \\ 0 & \text{otherwise.} \end{cases} \quad (14)$$

where the absolute value of $v(i)$ is used since its value could be either positive or negative depending on its directions. ξ denotes the drone speed (or acceleration in Eq. 15) along x-, y-, and z-axis in 3D space. Similarly, the acceleration cost f_a is computed by:

$$f_a = \sum_{\xi} \sum_{i=(pb-2)}^{(N-pb)} F_a(A_i)$$

$$F_a(A_i) = \begin{cases} (|a(i)| - A_{max})^2 & \text{if } |a(i)| \geq A_{max} \\ 0 & \text{otherwise.} \end{cases} \quad (15)$$

In practice, the following constraints and insights can be used to improve the usability of BladeView: (1) The inspection path should be parallel to a blade. (2) During the flight, the drone nose should always be perpendicular to the blade surface and perpendicular to the movement direction. (3) Due to the aerodynamic design of HAWT blades, the blade shape is gradually wider and thicker from tip to root (see Fig. 11). This shape, known as an airfoil, is like an uneven teardrop [50]. To ensure the quality of collected images, as detailed in Fig. 2, a simple yet practical strategy is: “fly a little farther (root section) to ensure the coverage, and fly a little closer (tip section) to ensure the blade resolution”. For instance, flying closer to the tip can provide the opportunity to increase the

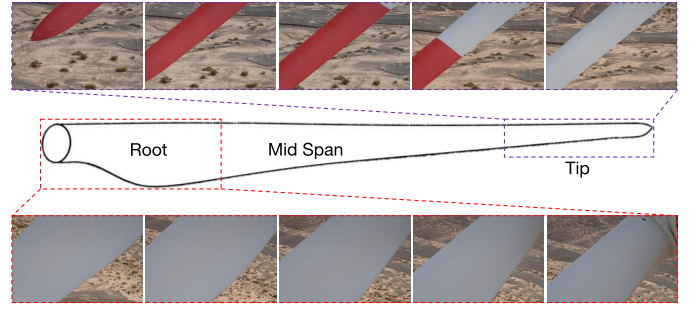


Fig. 11. Typical HAWT blade shape, and its tip and root sections.

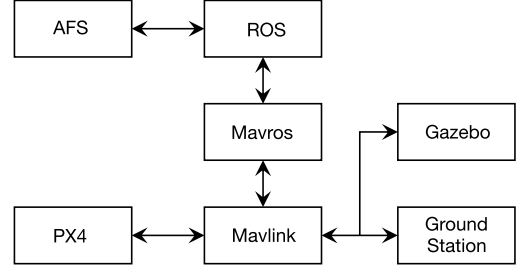


Fig. 12. Different modules in the simulated environment: Gazebo [21], ROS [51], PX4 [52] and our proposed AFS. Mavlink [53] is the message protocol. Mavros [54] is the message bridge between Mavlink and ROS.

resolution of the tip section where defects like edge erosion are frequent and require a more thorough assessment.

IV. EXPERIMENTS

We first detail the diverse types of environments involved in the BladeView evaluation. After that, we quantitatively study the performance of APC and AFS. We also evaluate BladeView by comparing it with existing methods in different environments. The limitations of our proposed method are discussed in the last part. Our experiments in Table II, IV and V were conducted along with commercial deployment from Clobotics Wind Service. Moreover, the ablation studies in Table III, and stress testing in Fig. 16 and 17 were addressed with the support of acknowledged research funds.

A. Environments

Our stated contributions are validated via flight tests in both simulated and real-world environments.

1) *Simulated Environment*: The simulated environment is established using Gazebo, ROS, and PX4. Gazebo [21] is an open-source 3D simulation software that has realistic physical environment characteristics for simulating drone motion characteristics. ROS [51] is a Robot Operating System that provides a communication architecture and combines various functional modules to work together. PX4 [52] is an open-source flight control platform that controls drone movement. Fig. 12 presents the details of our simulation architecture and the relationship between these different modules. In this paper, 4 Lenovo Y900 computers are used for the experiments. Each computer is equipped with Intel(R) Core(TM) i7-7700HQ (16 cores) CPU, GeForce 1060 GPU, 32G memory, 1TB storage, which is capable of simulating 100 flights per day. All

TABLE II

STATISTICS OF DIFFERENT ENVIRONMENTS FOR FIELD RUNS. WE PRESENT THE (MIN, MEAN, MAX) VALUES IN THE WIND SPEED, TURBINE HEIGHT, BLADE LENGTH, AND MINUTES PER TURBINE COLUMNS

Environments	Flights	Wind Speed (m/s)			Turbine Height (m)			Blade Length (m)			Minutes per Turbine			Success Rate (%)
		min	mean	max	min	mean	max	min	mean	max	min	mean	max	
Desert	803	3	9	14	39	51	75	19	40.3	53	19	22.5	25	98.0%
Shoreside	1108	5	10	15	46	71	82	25	48.3	61	20	24	27	97.0%
Hilly Area	1952	2	7	12	41	60	70.5	21	38.3	49.5	21	22.5	24	97.5%
Farmland	1331	3	8	13	43	59.6	72	22	39.3	51	21	22.5	24	99.0%
Suburb	903	2	8	13	41	61	73.5	21	39.6	52	20	23	25	98.5%
Gobi Desert	1935	3	9	14	41.5	62.6	77.5	19	41	56.5	20	22.5	26	97.0%
Offshore	1207	5	10	15	52	81.5	96	31	60.3	75	22	26	28	96.0%

TABLE III

ABLATION STUDY (ESTIMATION ERRORS) OF APC WITH TRIANGULATION $\Delta_{triangle}$, LINEAR FITTING Δ_{line} AND BOTH Δ_{result}

Parameters	Range	$\Delta_{triangle}$				Δ_{line}				Δ_{result}			
		min	max	mean	std	min	max	mean	std	min	max	mean	std
Turbine Height	60~200m	-3	3	-0.17	2.32	-1.5	1.5	0.13	1.13	-1	1	0.03	0.72
Turbine Position (m)	5~25m	-2	2	0.17	1.48	-2	2	0.20	1.57	-2	2	0.18	1.50
Turbine Position (°)	-30~30°	-5	5	-0.25	3.55	-5	5	0.08	3.63	-5	5	0.12	3.72
Blade Length*	30~100m	-1	1	-0.02	0.73	-1	1	-0.05	0.73	-1	1	-0.20	0.68
Blade Stop Angle	-90~30°	-3	3	-0.17	2.29	-1.5	1.5	-0.05	1.16	-1	1	0.02	0.73
Turbine Heading	0~360°	-5	5	-0.03	3.68	-1.5	1.5	0.03	1.15	-1	1	0.03	0.73
Turbine Radius	5~15m	-1	1	0.03	0.73	-1	1	0.05	0.79	-1	1	-0.02	0.75

*: The blade length is estimated after the first blade inspection. *m*: distance (meters), °: angle (degrees)

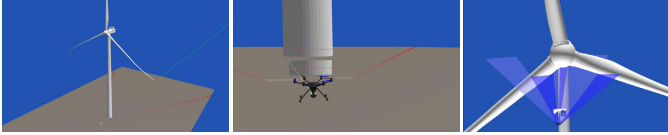


Fig. 13. Screenshots of the simulated environment.



Fig. 14. Our drone and testing in real-world environment.

the experiments are completed in 250 days. In total, we have applied around $4 \times 100 \times 250 = 100,000$ flight simulations in this environment.

The drone and turbine models are configured according to actual environments during the experiment. For instance, the turbine blades are stopped in the shape of a vertically flipped “Y”. The aircraft is a multi-rotor drone with both a camera and LiDAR mounted. The camera and LiDAR-covered areas are visualized for analysis (see Fig. 13). To assess the robustness of BladeView, we intentionally add interference factors from the real world to the simulator. For instance, GPS, wind, and magnetic compass interference could affect the drone and navigation stability, even causing the AFS to fail if the interference is substantial.

2) *Real-World Environment*: As presented in Fig. 14, real-world experiments are performed using a custom-built drone. Specifically, we use the six-rotor aircraft, DJI-M600, to carry the payload (camera, gimbal, onboard computer, and LiDAR).

Considering the FOV (Field of View) and detection range, the Velodyne VLP-16 LiDAR is selected. Note that though the solid-state LiDAR (CSL) has a smaller weight and consumes less power, its FOV is insufficient for our inspection tasks. Pixhawk is used as an onboard flight controller running the Ardupilot firmware. The off-board control and communication are carried out through the Intel NUC and ROS integration. A e-CAM-51 USB camera is equipped for capturing blade images. The motion planning algorithm in AFS is implemented in C++, and the Nlopt [55] solver is used for non-linear optimization. In practice, the drone can fly for more than 30 minutes on a full load, ensuring the completion of a single inspection task.

Table II details the statistics of real-world environments. With the commercial deployment of Clobotics Wind Service, we ran 9,239 flight tests on a total of 7 scenes. It can be observed that there are various challenges in these real-world environments. For instance, the sea is the most challenging (including both shoreside and offshore) since the wind speed is relatively high (5-15 meters per second). As a result, the time cost is the highest among all scenes: between 22-28 minutes. Moreover, it would also cost more to ship drones and pilots to offshore wind farms. Fig. 15 presents some sample images captured during the flight tests in these environments. Wind farms are located in China, Brazil, Denmark, and the USA. Due to privacy concerns, the wind farms’ metadata (location, company, etc.) are confidential.

B. Evaluation of APC

We first conduct an ablation study of APC based on the simulated environments since the turbine parameters can be easily



Fig. 15. Collected images from real-world environments: farmland, hilly area, suburb, the Gobi Desert, offshore, desert and shoreline.

TABLE IV

ESTIMATION ERROR OF TURBINE PARAMETERS BY APC IN REAL-WORLD ENVIRONMENTS. 1: TURBINE HEIGHT, 2: TURBINE POSITION (m), 3: TURBINE POSITION (°), 4: BLADE LENGTH, 5: BLADE STOP ANGLE, 6: TURBINE HEADING, 7: TURBINE RADIUS

	Desert				Shoreside				Hilly Area				Farmland				Suburb				Gobi Desert				Offshore			
	min	max	mean	std	min	max	mean	std	min	max	mean	std	min	max	mean	std	min	max	mean	std	min	max	mean	std	min	max	mean	std
1	-0.5	2	0.96	0.8	-0.8	2.1	1.20	1.2	-0.5	2.0	0.88	1.0	-0.4	2.1	0.80	1.0	-0.4	2.2	0.80	0.9	-0.8	2.0	1.04	1.0	-1.2	4.0	1.44	1.8
2	-2.0	3.0	0.61	1.9	-2.4	4.3	0.70	2.6	-1.9	2.5	0.59	1.6	-1.8	2.5	0.56	1.6	-1.8	2.5	0.55	1.7	-2.0	3.9	0.64	2.2	-2.5	4.7	0.66	2.9
3	-5.2	7.8	4.50	5.0	-5.8	13.0	5.40	7.2	-5.0	7.1	3.80	4.5	-4.6	7.0	3.40	4.4	-4.4	6.9	3.10	4.4	-5.5	9.9	4.90	5.6	-5.9	13.4	5.40	7.1
4	-3.0	4.1	1.90	2.5	-4.7	6.2	2.40	4.3	-2.7	3.9	1.74	2.3	-2.4	3.6	1.57	2.1	-2.4	3.8	1.57	2.2	-3.7	6.0	2.20	3.8	-4.3	5.9	2.30	3.8
5	-4.1	6.2	3.30	3.8	-6.8	10.2	3.90	6.6	-3.8	6.4	2.60	4.1	-3.7	6.2	2.50	3.4	-3.7	6.0	2.00	3.3	-6.1	9.5	3.60	5.8	-7.2	9.8	4.20	6.5
6	-3.1	5.4	2.90	3.2	-5.8	8.2	4.80	6.0	-3.0	5.1	2.60	3.0	-2.8	5.0	2.20	3.0	-2.7	5.0	2.20	2.7	-5.1	7.7	3.70	4.6	-6.9	10.1	5.50	6.6
7	-1.0	1.3	0.40	0.8	-1.2	2.3	0.48	1.3	-0.9	1.1	0.37	0.8	-0.8	1.0	0.36	0.7	-0.9	0.9	0.35	0.7	-1.2	2.1	0.43	1.1	-1.3	2.1	0.45	1.2

configured in the simulator (see Fig. 12). In this paper, ablation study means investigating the performance of APC by removing certain components (aka. $\Delta_{triangle}$ and Δ_{line}) to understand the contribution of the component to the overall APC. There are two steps for APC evaluation: Virtual Turbine Simulation and Virtual Turbine Inspection. (1) Virtual Turbine Simulation: the turbine parameters are randomly updated within each range (the range of each parameter is detailed in the second column of Table III). In total, 90 virtual turbines are simulated. In other words, these virtual turbines have different parameters (ground truth), and some parameters (turbine height and blade length) are intentionally set to be more extreme than real ones to address stress testing. For instance, the height of one virtual turbine is 200 meters, around 104 meters higher than the maximum in Table II (offshore, 96 meters). (2) Virtual Turbine Inspection: each inspection is independent and defined as an action from virtual drone takeoff to landing. In an inspection, APC estimates the turbine parameters in parallel, followed by our calculation of deviations from the ground truth. Thus, there is no estimation error correction since each inspection is independent. We collect virtual inspection results for the statistics, and the min/max deviations are detailed in Table III. In each simulation, we systematically evaluate the performance of parameter estimation by eliminating triangulation $\Delta_{triangle}$ and linear fitting Δ_{line} algorithms. The combination of both $\Delta_{triangle}$ and Δ_{line} (i.e. Δ_{result}) are also assessed for comparison. The estimation error of Turbine Height, Blade Length, and Turbine Radius means the deviation (in meters) between the estimated value (from APC) and the ground truth (real height, length, and radius). For instance, turbine height $\Delta_{triangle} = -3$ means that the estimated height from the triangulation algorithm is 3 meters shorter than the real turbine height. Blade length $\Delta_{line} = 1$ means that the estimated length from the linear fitting algorithm is 1 meter longer than the real blade length. Similarly, the estimation error of the Turbine Position means the deviation (in meters and degrees) of the estimated turbine position relative to the drone take-off point.

m and $^\circ$ denote the distance and direction, respectively. The estimation error of Blade Stop Angle and Turbine Heading means the deviation (in degrees) of the estimated value (from APC) and the ground truth (real blade stop and turbine heading angles).

In general, Δ_{result} (fusion of $\Delta_{triangle}$ and Δ_{line}) yields better results than the isolated $\Delta_{triangle}$ (triangulation algorithm) and Δ_{line} (linear fitting algorithm). We find that there is noticeable influence on the estimation performance from both triangulation $\Delta_{triangle}$ and linear fitting Δ_{line} ; $\Delta_{triangle}$ is more significant, particularly on Turbine Height and Blade Stop Angle. With the involvement of linear fitting Δ_{line} , the Turbine Height, Blade Stop Angle, and Turbine Heading estimation errors are significantly reduced. Thus, Δ_{result} is feasible for a more robust flight test in real-world environments. Note that in practice, $\Delta_{triangle}$ could be removed if (1) the onboard computational resource is limited, and (2) in low wind speed scenes like farmland and suburb. However, both $\Delta_{triangle}$ and Δ_{line} are required in offshore scenarios to ensure the success rate of inspection on large wind speed, turbine height and blade length.

We also evaluate the performance of APC in real-world environments. Based on the flight tests in Table II, we report the estimation error of the turbine parameters in 7 different environments. As detailed in Table IV, we can observe that the estimation errors (mean value) in the shoreside and offshore scenes are the highest, around 10%-25% higher than others. The main reason is that the offshore wind speed is 5-15 meters per second, much faster than that in other scenes (see Table II). As a result, it is challenging to maintain the performance of APC in such harsh conditions. Nevertheless, we find that the estimated parameters are within a tolerable accuracy range for practical inspection purposes. The results in Table III (simulated environment) are slightly better than the mean errors in Table IV (real-world environment). Expectedly, real-world interferences, particularly those caused by wind and ambient light can further complicate the calculations in the

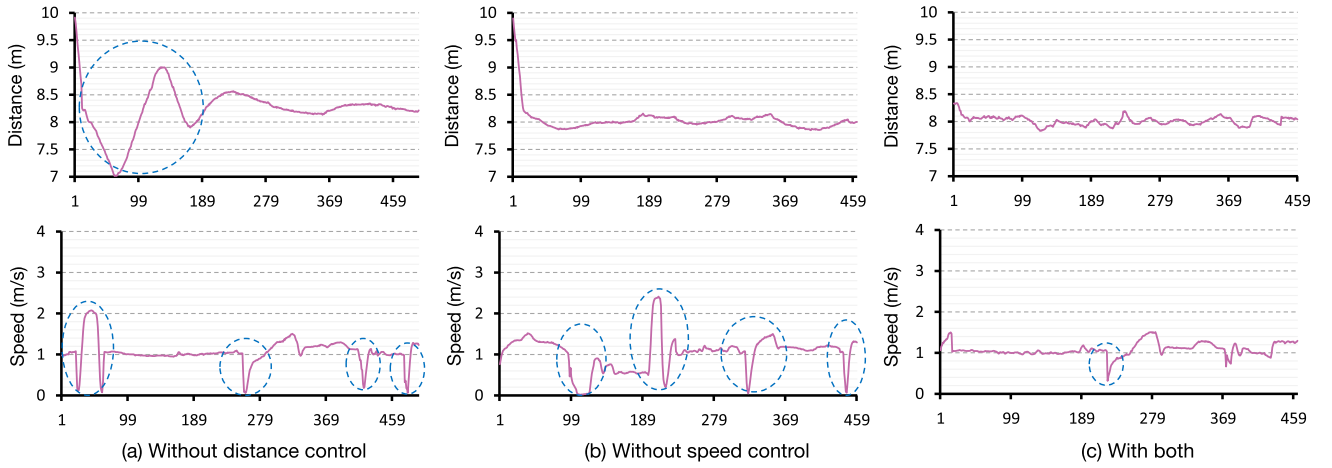


Fig. 16. Ablation study of distance control d_{cost} and speed control F_v in Eq. 9. The horizontal axis denotes time (seconds).

APC. For instance, ambient light disturbances could bring about noises that negatively influence the generation of LiDAR point clouds.

C. Evaluation of AFS

To evaluate the effectiveness of AFS for a smooth inspection, we conduct an ablation study on the distance control d_{cost} (Eq. 9) and speed control F_v (Eq. 14) in the simulated environment. For a fair comparison, the environmental conditions are kept the same while d_{cost} and F_v are eliminated independently. In Fig. 16, the drone speed and distance are plotted along the time axis (seconds). Since both drone speed and distance appear to be closely correlated (see Eq. 13 and 14), eliminating the distance control d_{cost} , resulted in varying inconsistencies in both distance and speed (areas marked by blue dotted ellipses in Fig. 16 (a)). In contrast, if we eliminate the speed control F_v , the drone speed becomes unstable (ranging from 0 to 2.6 m/s) while the distance remains relatively smooth (see Fig. 16 (b)). In other words, in addition to the expected influence on distance, the distance control term also shows substantial influence on drone speed. In Fig. 16 (c), both speed and distance are smooth if d_{cost} and F_v are applied, indicating the importance of controlling both these parameters. This setting will be the default for the remaining experiments in this paper.

To evaluate the robustness of AFS under various interferences, we performed 100 flight tests in the simulated environment with randomly varying interference factors in each trial: GPS, IMU, and wind speed. Then, the mean values of the drone speed and distance are calculated. As presented in Fig. 17, we find that AFS is robust to the aforementioned interferences. The distance and speed are primarily maintained within 8-23 m and 0.8-1.9 m/s, respectively. Note that the broad change range (8 to 23 meters) is partly due to different pre-defined distances in the blade root and tip (see Fig. 11). Thus, image resolution and defect detection accuracy are ensured. However, as marked by ellipses in Fig. 17, there are still some flight risks under strong interferences (this is discussed in Section IV-E).

D. Evaluation of BladeView

Based on the real-world environments, the global performance of BladeView is detailed in Table II (the rightmost

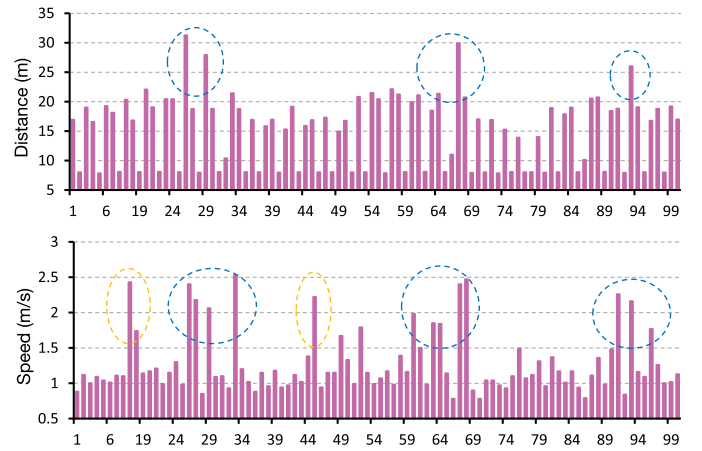


Fig. 17. Evaluation of AFS (drone speed and distance) under random interferences. The horizontal axis denotes the index of each test. Blue ellipses denote the correlated drone speeds and distances. Orange ellipses indicate the uncorrelated speeds (not as much increase in distance). Best viewed in color.

column). As introduced in Section I, a successful inspection is defined as efficient, full blade covering and successful image collection between take-off and return to launch. For instance, an inspection will be considered a failure for one of the following reasons: (1) duration longer than 30 minutes, (2) a particular blade region is not covered, (3) the collected images do not meet the requirements, (4) collision, and (5) pilot takeover is required. While (2), (3), and (4) are easy to understand, we mainly focus on (1) and (5). For (1), the maximum inspection time is 30 minutes due to the stated lifespan of drone batteries, particularly for large turbines (see Turbine Height and Blade Length in Table III, possible to resume the inspection following a battery swap). In practice, pilot takeover occurs typically when the inspection is unfinished (due to, e.g., low battery, sudden weather change, etc.), and the drone returns to launch with pilot control. While BladeView aims to finish the whole inspection automatically (except the requirement for unanticipated intervention to correct course deviation or prevent a crash constitutes failure), we treat (5) as a failure since the inspection is not finished.

We find that the success rates across all seven scenes is between 96% to 99%, more reliable than several reported

TABLE V

QUANTITATIVE AND COMPREHENSIVE EVALUATION OF OUR PROPOSED BLADEVIEW. IT WAS REPEATED 100 TIMES ON A REAL-WORLD WIND TURBINE TO CALCULATE THE SUCCESS RATE AND THE VALUES OF THREE OTHER METRICS

Method	Success Rate	Power Used (%)				Flight Time (minutes)				Photo Coverage (%)			
		min	max	mean	std	min	max	mean	std	min	max	mean	std
BladeView	96.8%	73%	88%	80%	0.054	16	23	20	2.702	92%	97%	95%	0.019

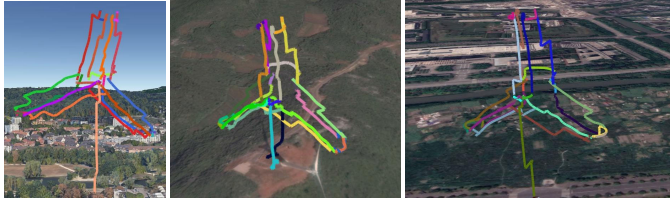


Fig. 18. Real-world inspection paths. For better visualization, the real paths are marked in different colors (according to blades and edges), and projected onto the 3D space of Google Maps.

manual inspection methods [19], [56]. Remarkably, the success rate in farmland is around 99%. Even in harsh environments like shoreside and offshore, the success rate is still between 96% to 97%. Based on our real-world experience on wind-turbine services [6], manual operation on drone-based wind turbine inspection is challenging in practice, even when operated by experienced pilots – this strictly defined success rate can be close to 0%. Note that our claims are only perceived as unduly critical of current industrial practice. A large number of airborne inspection processes are carried out by commercial operators on a daily basis worldwide using manually piloted drones. Here, BladeView is reportedly more reliable, practical, and efficient; each inspection only takes 19 to 28 minutes per turbine (see the penultimate column in Table II). According to the data from Salary.com (2024), the salaries of drone pilots are quantified at three levels: entry, experienced, and senior. Entry-level pilots make an average of \$41.21 per hour, while experienced pilots make \$53.91 per hour on average. Senior-level pilots make the most at an average of \$74.04 per hour. In our case, an entry-level pilot is enough for BladeView triggering and supervision in practice.

Based on a standard wind turbine (height: 85m, blade length: 65m) in the real-world environment (Gobi desert in Gansu Province, China), we also quantitatively evaluate the performance of BladeView. To do so, we conducted the inspection 100 times independently to offset the influence of different wind speeds. Four metrics (success rate, mean power consumption values, inspection time, and photo coverage) are used for the final analysis. The whole experiment took around three days to ensure the diversity of wind speed. Thus, this experiment is more similar to “long stability testing”. As detailed in Table V, our proposed BladeView has high success rate and photo coverage within reasonable power and time consumption. In other words, our proposed BladeView is practical and efficient.

To visually evaluate the inspection performance, we analyse the stability of real-world inspection paths and the quality of collected images. For the first factor, as presented in Fig. 18, the inspection paths are marked with different colors



(a) Blade images



(b) Full blade after stitching

Fig. 19. Blade images taken by BladeView and the stitching result.

(according to blades and edges) and projected onto the 3D space of Google Maps. We can observe that the inspection paths obtained in different environments are smooth, constant, and similar to the pre-defined ones in Fig. 2. Thus, our proposed inspection path is consistent and standardized, making it suitable for HAWT in most environments. Here, we report the trajectory errors (minimal, maximum, mean, standard deviation) of three monitoring points (tip, middle, root) of a blade: Tip [min, max, mean, std] is [0.11, 0.53, 0.25, 0.20], Middle [min, max, mean, std] is [0.31, 0.82, 0.50, 0.49], Root [min, max, mean, std] is [0.54, 1.03, 0.64, 0.61]. In practice, Blade 1, 2, and 3 (see Fig. 2) have similar trajectory errors. The trajectory errors near the blade root regions are slightly larger than the middle, followed by the tip. It is understandable since the blade tip is thinner, simpler and has less LiDAR interference. As a result, the fitted path from LiDAR is more accurate (see Section III-B). In contrast, the root region is thicker and more complicated (joint of three roots), with more noise in the fitting from LiDAR.

On the image quality, Fig. 19 (a) presents a set of images of a single blade collected with BladeView. We can observe the following characteristics of these images: (1) Stitching is easy since the blade region is strictly located along the image diagonal. (2) Easy to analyze (particularly defects) since the images are of high quality. (3) Full blade covering from root to tip, no missing regions. Fig. 19 (b) presents the stitched image based on [57]. In practice, the stitched image in Fig. 19 (b) is used for full blade visualization, defect localization, and defect analysis in wind farm management systems [58]. Here, we would like to provide a preface to the background of the stitching evaluation. We can not quantitatively compare



Fig. 20. Broken drones from our flight tests, mostly due to collision.

the stitching results without clarifying the stitching algorithm. Due to wide parallax and texture-poor features, blade stitching is challenging, and stitching accuracy with existing algorithms [59], [60] cannot reach the industrial requirements (e.g., 8 pixels error within 5400×3600 resolution). For this, we introduced a coarse-to-fine blade stitching strategy, which yields 2-3 times better accuracy than existing methods. This work has been published in [57]. Thus, we visualize (qualitative evaluation) the stitching result to demonstrate the usability of our collected data with BladeView. As presented in Fig 3, the collected images are mostly unfit (partial-missing, blurry, and wrong direction, etc.) for the purpose. Our proposed stitching approach in [57] can not be directly applied in such a case. It should be noted that the specific conditions (8 pixels error within 5400×3600 resolution) are commercial requirements (e.g., GoldWind, Energy China, Eurus Energy, and Ocean Breeze Energy GmbH, etc.) and may not be generically applicable across different imaging hardware and applications.

E. Limitations

Fig. 20 presents drone casualty cases during our testing in real-world environments. BladeView is limited in two aspects: (1) The drone distance and speed are not smooth enough during the path transition from the pressure side to the trailing edge. (2) Bladeview requires the intervention of the pilot (switching from automatic mode to manual) in some extreme conditions, such as severe GPS and magnetic compass interference, and strong gusts (sudden strong wind) above the tolerance level. All these aspects could potentially lead to a collision. To solve these problems, one possible circumvention is to check the weather conditions before the inspection takes place. Another idea involves developing a real-time warning model for notification under extreme and unsafe flight conditions. Based on our field run experiences, the maximum interference that BladeView can carry is as follows: the maximum wind speed is $10m/s$ (less than the Strong Breeze), the maximum turbine height is $200m$, the maximum battery support time is 30 minutes, and the maximum bias of GPS and IMU is $10m$ and $10cm$, respectively.

V. CONCLUSION

This paper introduces a simple yet efficient method, BladeView, for drone-based wind turbine inspection. BladeView comprises two major components: Automatic Parameter Calculation (APC) and Automatic Flight System (AFS). APC aims to calculate the parameters of a wind turbine, thereby automatically adapting different turbine types, stop angles, and

relative positions with the drone. AFS is used for planning an optimal three-blade-covered inspection path within dynamic flight corridors, in compliance with the turbine parameters calculated by APC. The usability, efficiency, and robustness of BladeView are validated through rigorous flight tests conducted in simulated (100,000 tests) and real-world (9,239 tests) environments. Our comprehensive experiments also show that BladeView yields a 6.7% improvement in success rate than the classic PID method, while the overall power consumption is reduced by around 15%. In the future, we will close the loop of wind services, including blade inspection, defect analysis, and blade repair. For instance, automatically repair a blade based on the metrics of detected defects.

REFERENCES

- [1] M. Car, L. Markovic, A. Ivanovic, M. Orsag, and S. Bogdan, "Autonomous wind-turbine blade inspection using LiDAR-equipped unmanned aerial vehicle," *IEEE Access*, vol. 8, pp. 131380–131387, 2020.
- [2] F. Porté-Agel, M. Bastankhah, and S. Shamsoddin, "Wind-turbine and wind-farm flows: A review," *Boundary-Layer Meteorol.*, vol. 174, no. 1, pp. 1–59, Jan. 2020.
- [3] F. P. G. Márquez and A. M. P. Chacón, "A review of non-destructive testing on wind turbines blades," *Renew. Energy*, vol. 161, pp. 998–1010, Dec. 2020.
- [4] I. Gohar, A. Halimi, J. See, W. K. Yew, and C. Yang, "Slice-aided defect detection in ultra high-resolution wind turbine blade images," *Machines*, vol. 11, no. 10, p. 953, Oct. 2023.
- [5] P. Nooralishahi et al., "Drone-based non-destructive inspection of industrial sites: A review and case studies," *Drones*, vol. 5, no. 4, p. 106, Sep. 2021.
- [6] *Clobotics*. Accessed: Apr. 6, 2024. [Online]. Available: <https://clobotics.com/wind/>
- [7] *Skydio*. Accessed: Apr. 6, 2024. [Online]. Available: <https://www.skydio.com/>
- [8] *Sulzer Schmid*. Accessed: Apr. 6, 2024. [Online]. Available: <https://www.sulzerschmid.ch/offering/>
- [9] *Perceptual Robotics*. Accessed: Apr. 6, 2024. [Online]. Available: <https://www.perceptual-robotics.com/solutions/>
- [10] *Skyspecs*. Accessed: Apr. 6, 2024. [Online]. Available: <https://skyspecs.com/>
- [11] M. Memari, P. Shakya, M. Shekaramiz, A. C. Seibi, and M. A. S. Masoum, "Review on the advancements in wind turbine blade inspection: Integrating drone and deep learning technologies for enhanced defect detection," *IEEE Access*, vol. 12, pp. 33236–33282, 2024.
- [12] P. Durdevic, D. Ortiz-Arroyo, and Z. Yang, "LiDAR assisted camera inspection of wind turbines: Experimental study," in *Proc. 1st Int. Conf. Electr., Control Instrum. Eng. (ICECIE)*, Nov. 2019, pp. 1–7.
- [13] M. Stokkeland, K. Klausen, and T. A. Johansen, "Autonomous visual navigation of unmanned aerial vehicle for wind turbine inspection," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2015, pp. 998–1007.
- [14] L. Wang and Z. Zhang, "Automatic detection of wind turbine blade surface cracks based on UAV-taken images," *IEEE Trans. Ind. Electron.*, vol. 64, no. 9, pp. 7293–7303, Sep. 2017.
- [15] O. Moolan-Feroze, K. Karachalios, D. N. Nikolaidis, and A. Calway, "Simultaneous drone localisation and wind turbine model fitting during autonomous surface inspection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 2014–2021.
- [16] B. E. Schäfer, D. Picchi, T. Engelhardt, and D. Abel, "Multicopter unmanned aerial vehicle for automated inspection of wind turbines," in *Proc. 24th Medit. Conf. Control Autom. (MED)*, Jun. 2016, pp. 244–249.
- [17] S. Du and C. Tu, "Power line inspection using segment measurement based on HT butterfly," in *Proc. IEEE Int. Conf. Signal Process., Commun. Comput. (ICSPCC)*, Sep. 2011, pp. 1–4.
- [18] O. Moolan-Feroze, K. Karachalios, D. N. Nikolaidis, and A. Calway, "Improving drone localisation around wind turbines using monocular model-based tracking," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2019, pp. 7713–7719.

- [19] T. M. Letcher, *Wind Energy Engineering: A Handbook for Onshore and Offshore Wind Turbines*. New York, NY, USA: Academic, 2017.
- [20] A. Kulsinskas, P. Durdevic, and D. Ortiz-Arroyo, "Internal wind turbine blade inspections using UAVs: Analysis and design issues," *Energies*, vol. 14, no. 2, p. 294, Jan. 2021.
- [21] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, Feb. 2004, pp. 2149–2154.
- [22] A. Shihavuddin et al., "Wind turbine surface damage detection by deep learning aided drone inspection analysis," *Energies*, vol. 12, no. 4, p. 676, Feb. 2019.
- [23] C. Martinez et al., "Predicting wind turbine blade erosion using machine learning," *SMU Data Sci. Rev.*, vol. 2, no. 2, p. 17, 2019.
- [24] D. Denhof, B. Staar, M. Lütjen, and M. Freitag, "Automatic optical surface inspection of wind turbine rotor blades using convolutional neural networks," *Proc. CIRP*, vol. 81, pp. 1166–1170, Jan. 2019.
- [25] R. A. S. Fernández, S. Dominguez, and P. Campoy, "L₁ adaptive control for wind gust rejection in quad-rotor UAV wind turbine inspection," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2017, pp. 1840–1849.
- [26] N. Simon et al., "FlowDrone: Wind estimation and gust rejection on UAVs using fast-response hot-wire flow sensors," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 5393–5399.
- [27] S. S. Mansouri, C. Kanellakis, E. Fresk, D. Kominak, and G. Nikolakopoulos, "Cooperative coverage path planning for visual inspection," *Control Eng. Pract.*, vol. 74, pp. 118–131, May 2018.
- [28] C. Kanellakis, E. Fresk, S. S. Mansouri, D. Kominak, and G. Nikolakopoulos, "Towards visual inspection of wind turbines: A case of visual data acquisition using autonomous aerial robots," *IEEE Access*, vol. 8, pp. 181650–181661, 2020.
- [29] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon path planning for 3D exploration and surface inspection," *Auto. Robots*, vol. 42, no. 2, pp. 291–306, Feb. 2018.
- [30] W. Jing, D. Deng, Z. Xiao, Y. Liu, and K. Shimada, "Coverage path planning using path primitive sampling and primitive coverage graph for visual inspection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 1472–1479.
- [31] J. A. Guerrero and Y. Bestaoui, "UAV path planning for structure inspection in windy environments," *J. Intell. Robot. Syst.*, vol. 69, nos. 1–4, pp. 297–311, Jan. 2013.
- [32] W. Jing, D. Deng, Y. Wu, and K. Shimada, "Multi-UAV coverage path planning for the inspection of large and complex structures," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 1480–1486.
- [33] S. Vanneste, B. Bellekens, and M. Weyn, "3DVFH+: Real-time three-dimensional obstacle avoidance using an OctoMap," in *Proc. MORSE STAF*, 2014, pp. 91–102.
- [34] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *Int. J. Robot. Res.*, vol. 29, no. 5, pp. 485–501, Apr. 2010.
- [35] B. T. Lopez and J. P. How, "Aggressive 3-D collision avoidance for high-speed navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2017, pp. 5759–5765.
- [36] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3529–3536, Oct. 2019.
- [37] L. Han, F. Gao, B. Zhou, and S. Shen, "FIESTA: Fast incremental Euclidean distance fields for online motion planning of aerial robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 4423–4430.
- [38] L. Gooijaers, "Wind optimised trajectory generator for automated wind turbine inspector drones," Master's thesis, Fac. Aerosp. Eng., TU Delft Aerosp. Eng., Delft, The Netherlands, Feb. 2021.
- [39] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Auto. Robots*, vol. 34, no. 3, pp. 189–206, Apr. 2013.
- [40] B. LeBlanc, C. Niezrecki, P. Avitabile, J. Sherwood, and J. Chen, "Surface stitching of a wind turbine blade using digital image correlation," in *Topics in Modal Analysis II*, vol. 6. New York, NY, USA: Springer, 2012, pp. 277–284.
- [41] R. Schnabel, R. Wahl, and R. Klein, "Efficient RANSAC for point-cloud shape detection," *Comput. Graph. Forum*, vol. 26, no. 2, pp. 214–226, 2007.
- [42] M. Oliveira and A. Marcal, "Clustering LiDAR data with K-means and DBSCAN," in *Proc. 12th Int. Conf. Pattern Recognit. Appl. Methods*, 2023, pp. 822–831.
- [43] S. J. Julier and J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, Mar. 2004.
- [44] F. Gao, L. Wang, B. Zhou, X. Zhou, J. Pan, and S. Shen, "Teach-repeat-replan: A complete and robust system for aggressive flight in complex environments," *IEEE Trans. Robot.*, vol. 36, no. 5, pp. 1526–1545, Oct. 2020.
- [45] P. Wang, Z. Man, Z. Cao, J. Zheng, and Y. Zhao, "Dynamics modelling and linear control of quadcopter," in *Proc. Int. Conf. Adv. Mech. Syst. (ICAMechS)*, Nov. 2016, pp. 498–503.
- [46] M. W. Mueller, M. Hehn, and R. D'Andrea, "A computationally efficient motion primitive for quadcopter trajectory generation," *IEEE Trans. Robot.*, vol. 31, no. 6, pp. 1294–1310, Dec. 2015.
- [47] K. Qin, "General matrix representations for B-splines," *Vis. Comput.*, vol. 16, nos. 3–4, pp. 177–186, 2000.
- [48] S. Quinlan and O. Khatib, "Elastic bands: Connecting path planning and control," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 1993, pp. 802–807.
- [49] Z. Zhu, E. Schmerling, and M. Pavone, "A convex optimization approach to smooth trajectories for motion planning with car-like robots," in *Proc. IEEE Conf. Decis. Control*, Dec. 2015, pp. 835–842.
- [50] P. J. Schubel and R. J. Crossley, "Wind turbine blade design," *Energies*, vol. 5, no. 9, pp. 3425–3449, 2012.
- [51] M. Quigley et al., "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Softw.*, 2009, vol. 3, no. 3.2, p. 5.
- [52] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2015, pp. 6235–6240.
- [53] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, "Micro air vehicle link (MAVlink) in a nutshell: A survey," *IEEE Access*, vol. 7, pp. 87658–87680, 2019.
- [54] P. Smyczynski, L. Starzec, and G. Granasik, "Autonomous drone control system for object tracking: Flexible system design with implementation example," in *Proc. 22nd Int. Conf. Methods Models Autom. Robot. (MMAR)*, Aug. 2017, pp. 734–738.
- [55] S. G. Johnson and J. Schueller, "NLopt: Nonlinear optimization library," *Astrophys. Source Code Library*, p. 2111, Nov. 2021.
- [56] K. K. Poleo, W. J. Crowther, and M. Barnes, "Estimating the impact of drone-based inspection on the Levelised cost of electricity for offshore wind farms," *Results Eng.*, vol. 9, Mar. 2021, Art. no. 100201.
- [57] C. Yang, X. Liu, H. Zhou, Y. Ke, and J. See, "Towards accurate image stitching for drone-based wind turbine blade inspection," *Renew. Energy*, vol. 203, pp. 267–279, Feb. 2023.
- [58] B. Mohandes, M. Wahbah, M. S. E. Moursi, and T. H. M. El-Fouly, "Renewable energy management system: Optimum design and hourly dispatch," *IEEE Trans. Sustain. Energy*, vol. 12, no. 3, pp. 1615–1628, Jul. 2021.
- [59] M. Brown and D. G. Lowe, "Automatic panoramic image stitching using invariant features," *Int. J. Comput. Vis.*, vol. 74, no. 1, pp. 59–73, Aug. 2007.
- [60] K. Lin, N. Jiang, L. F. Cheong, M. N. Do, and J. Lu, "SEAGULL: Seam-guided local alignment for parallax-tolerant image stitching," in *Proc. Eur. Conf. Comput. Vis.*, Oct. 2016, pp. 370–385.



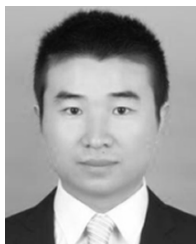
Cong Yang received the Ph.D. degree in computer vision and pattern recognition from the University of Siegen, Germany, in 2016. He has been an Associate Professor with Soochow University since 2022. Before that, he was a Post-Doctoral Researcher with the MAGRIT Team, INRIA, France. Later, he worked scientifically and led the computer vision and machine learning teams at Clobotics and Horizon Robotics. His research interests include computer vision, pattern recognition, and their interdisciplinary applications.



Hua Zhou received the bachelor's degree in mechanical engineering from Northeast University, China, in 2014, and the master's degree in aeronautical and astronautical science and technology from Shanghai Jiao Tong University in 2017. He has worked on motion plans and LIDAR simultaneous localization and mapping. His research interests include the control and trajectory generation for multirotor vehicles and obstacle avoidance with less fuel and time.



Marcin Grzegorzek received the Doctor of Engineering degree in pattern recognition from the University of Erlangen–Nürnberg in 2007. Later, he worked scientifically at the Queen Mary University of London and the University of Koblenz and Landau. From 2010 to 2018, he was an Assistant Professor of pattern recognition with the University of Siegen. He is currently a Professor of medical informatics with the University of Luebeck, with a research focus on medical data science.



Xun Liu received the B.E. degree in software engineering from Northeast Normal University, China, in 2010, and the Ph.D. degree in computer science from Zhejiang University in 2015. He is currently a Computer Vision Expert on deep learning solutions in the company. His research interests include computer vision, machine learning, deep learning, perceptive algorithms, and big data analysis.



Zeyd Boukhers received the Ph.D. degree in pattern recognition from the University of Siegen in 2017. He is currently the co-lead of the FAIR Data and Distributed Analytics Research Group, Fraunhofer Institute for Applied Information Technology. In addition, he is a Senior Researcher with the University Hospital of Cologne. He has authored over 30 papers in esteemed journals and conferences. His research interests include machine learning, data science, computer vision, and natural language processing.



Yan Ke received the bachelor's degree in computer science, the master's degree in electrical and computer engineering, and the Ph.D. degree in computer science from Carnegie Mellon University, in 2008. He is currently the Co-Founder and the CTO of Clobotics, a computer vision company serving the retail and wind industries. He is an expert in computer vision, data mining, machine learning, and distributed systems. Previously, he spent eight years at Microsoft leading the Bing Entity Understanding Group until 2016. He has published in top international computer vision conferences, such as IEEE CVPR and ICCV. He holds eight U.S. patents.



Tao Chen received the B.Sc. degree in mechanical design, manufacturing and automation, the M.Sc. degree in mechatronic engineering, and the Ph.D. degree in mechatronic engineering from Harbin Institute of Technology, Harbin, China, in 2004, 2006, and 2010, respectively. He is currently a Visiting Scholar with the National University of Singapore in 2018. He is currently a Professor with the School of Future Science and Engineering, Soochow University, Suzhou, China. His research interests include MEMS, sensors, and actuators.



Bo Gao received the bachelor's and master's degrees in computer science from Shanghai Jiao Tong University in 2005 and 2008, respectively. He is currently the Vice President of research and engineering with Clobotics and is responsible for applying computer vision, NLP, AIoT, and big data technologies to retail and ESG industries. Before that, he was with Morgan Stanley, EMC, Yitu, and other companies responsible for research and technology development in artificial intelligence, storage, and big data processing.



John See received the B.Eng., M.Eng., and Ph.D. degrees from Multimedia University, Malaysia. He is currently an Associate Professor with the School of Mathematical and Computer Sciences, Heriot-Watt University (Malaysia Campus). He is also an award recipient of the Belt and Road Initiative Young Scientist Exchange Fellowship, where he was a Visiting Research Fellow with Shanghai Jiao Tong University, China. He regularly publishes in reputable journals and major computer vision conferences. His research interests include computer vision and image processing, particularly in the emerging fields of facial micro-expressions, affective computing, computational aesthetics, and deep learning. He is currently an Associate Editor of IEEE TRANSACTIONS ON MULTIMEDIA, *EURASIP Journal on Image and Video Processing*, and *Frontiers in Signal Processing*.